



Pandas Cheatsheet

(data)praktycznie i bez chaosu · workflow każdego projektu — od pliku do wniosków

01 Wczytywanie · 02 Inicjalna analiza · 03 Czyszczenie i przygotowanie · 04 Eksploracja (EDA)

01 · Wczytywanie danych

Import bibliotek

```
import pandas as pd
import numpy as np
```

Wczytanie pliku

```
df = pd.read_csv("dane.csv") # CSV
df = pd.read_csv("dane.csv", sep=";", decimal=",") # europejski format
df = pd.read_excel("dane.xlsx", sheet_name=0) # Excel
df = pd.read_parquet("dane.parquet") # szybki format kolumnowy
df = pd.read_json("dane.json") # JSON
```

Przydatne argumenty read_csv

```
pd.read_csv("dane.csv",
            usecols=["a", "b"], # tylko wybrane kolumny
            nrows=1000, # tylko N pierwszych wierszy (podgląd)
            parse_dates=["data"], # od razu parsuj daty
            dtype={"id": str}) # wymuś typy
```

02 · Inicjalna analiza danych

Podstawowe informacje

```
df.shape           # liczba wierszy i kolumn
df.columns         # nazwy kolumn
df.dtypes          # typy danych kolumn
df.info()          # typy + non-null + pamięć (najw
ażniejsze!)
df.memory_usage(deep=True) # zużycie pamięci per kolumna
```

Podgląd wartości

```
df.head()          # pierwsze 5 wierszy
df.tail()          # ostatnie 5 wierszy
df.sample(5)       # losowe 5 – lepsze niż head przy danych posor
towanych
```

Braki danych

```
df.isna().sum()    # liczba braków w każdej kolumni
e
df.isna().mean()   # udział braków (× 100 dla proce
ntów)
df[df.isna().any(axis=1)] # wiersze z jakimkolwiek brakiem
df.isna().sum().sum() # łączna liczba braków
```

Duplikaty

```
df.duplicated().sum() # liczba zduplikowany
ch wierszy
df[df.duplicated()]   # podgląd duplikatów
```

```
df.duplicated(subset=["kol1", "kol2"]) # duplikaty po wybranych kolumnach
```

Unikalność i licznosc

```
df["kol"].value_counts() # licznosc kazdej wartosci
df["kol"].value_counts(normalize=True) # udzial procentowy
df["kol"].value_counts(dropna=False) # uwzględnij NaN
df["kol"].nunique() # liczba unikalnych wartosci
df["kol"].unique() # lista unikalnych wartosci
```

Szybki opis statystyczny

```
df.describe() # kolumny numeryczne
df.describe(include="all") # wszystkie kolumny, tez tekstowe
```

Mini-checklista na start projektu

- ☐ `df.head()` — jak wygladaja dane?
- ☐ `df.shape` — ile tego jest?
- ☐ `df.info()` — typy i braki naraz
- ☐ `df.describe()` — rozklady wartosci
- ☐ `df.isna().mean()` — gdzie sa braki?
- ☐ `df.duplicated().sum()` — czy sa duplikaty?
- ☐ `df["k"].value_counts()` — co w kategoriach?
- ☐ `df.corr(numeric_only=True)` — co z czym?

03 · Czyszczenie i przygotowanie danych

Porządkowanie nazw kolumn

```
df.columns = df.columns.str.strip().str.lower() # spacje + małe litery
df = df.rename(columns={"stara": "nowa"})
```

Braki — uzupełnianie / usuwanie

```
df = df.dropna() # usuń wiersze z brakiem
df = df.dropna(subset=["wazna"]) # tylko gdy brak w tej kolumnie
df = df.fillna(0) # wypełnij stałą
df["x"] = df["x"].fillna(df["x"].median()) # liczbowe: mediana
df["k"] = df["k"].fillna(df["k"].mode()[0]) # kategoriyczne: moda
```

Duplikaty — usuwanie

```
df = df.drop_duplicates() # zamiast inplace=True
df = df.drop_duplicates(subset=["id"], keep="last") # zachowaj ostatni
```

Typy danych — konwersja

```
df["kol"] = df["kol"].astype(int)
df["data"] = pd.to_datetime(df["data"])
df["liczba"] = pd.to_numeric(df["liczba"], errors="coerce")
# błędy -> NaN
```

```
df["kat"] = df["kat"].astype("category") # oszczędza pamięć
```

Tekst — czyszczenie

```
df["kol"] = df["kol"].str.strip().str.lower()  
df["kol"] = df["kol"].str.replace(r"\s+", " ", regex=True) #  
spacje -> jedna
```

Daty — wydobywanie składników

```
df["data"] = pd.to_datetime(df["data"])  
df["rok"] = df["data"].dt.year  
df["miesiac"] = df["data"].dt.month  
df["dzien_tyg"] = df["data"].dt.day_name()
```

04 · Eksploracja danych (EDA)

Statystyki szczegółowe

```
df["x"].mean() # średnia  
df["x"].median() # mediana  
df["x"].mode() # moda  
df["x"].std() # odchylenie standardowe  
df["x"].skew() # skośność (asymetria rozkładu)  
df["x"].kurt() # kurtoza (spłaszczenie rozkładu)
```

Wartości odstające (IQR)

```
q1 = df["x"].quantile(0.25)  
q3 = df["x"].quantile(0.75)
```

```
iqr = q3 - q1
out = df[(df["x"] < q1 - 1.5*iqr) | (df["x"] > q3 + 1.5*iqr)]
```

Korelacje

```
df.corr(numeric_only=True)          # numeric_only WYMAGANE w pandas 2.x
df.corr(numeric_only=True)["x"]     # korelacja względem jednej kolumny
```

Grupowanie i agregacja

```
df.groupby("kat")["x"].mean()
df.groupby("kat").agg(
    srednia=("x", "mean"),
    suma=("x", "sum"),
    liczba=("x", "count"))
```

Tabele krzyżowe i przestawne

```
pd.crosstab(df["a"], df["b"])
df.pivot_table(index="region", columns="rok",
               values="sprzedaz", aggfunc="sum")
```

Selekcja i filtrowanie

```
df.loc[df["wiek"] > 30]
df.loc[(df["wiek"] > 30) & (df["miasto"] == "Warszawa")] # & oraz |
df.loc[df["miasto"].isin(["Warszawa", "Kraków"])]
df.iloc[0:5, 0:3]                                         # po pozycjach
df.query("wiek > 30 and miasto == 'Warszawa'")          # czytelniej
```

Sortowanie

```
df.sort_values("x", ascending=False).head(10)    # top 10 / ek  
strema  
df.sort_values(["a", "b"], ascending=[True, False])
```

Tworzenie kolumn

```
df["nowa"] = df["a"] + df["b"]  
df["grupa"] = df["wiek"].apply(lambda v: "dorosly" if v >= 18  
else "dziecko")  
df = df.assign(  
    zysk=lambda d: d["przychod"] - d["koszt"],  
    marza=lambda d: d["zysk"] / d["przychod"])
```

Zapis wyników

```
df.to_csv("wynik.csv", index=False)  
df.to_excel("wynik.xlsx", index=False)  
df.to_parquet("wynik.parquet")
```

Przebranżowienie • Data Analyst